

PYTHON OOPS

117 Python OOP Interview Questions with Trainer-Style Answers

Classes • Objects • Encapsulation • Inheritance • Polymorphism • Abstraction • Exceptions • File Handling • Serialization

1. What is a class?

Answer: A class is a blueprint for creating objects. It defines data through attributes and behavior through methods.
Example: class Car: pass creates a class named Car.

2. What is an object?

Answer: An object is a runtime instance of a class. It has identity, state (attributes), and behavior (methods).

3. What is the difference between a class and an object?

Answer: A class is a blueprint or definition; an object is a concrete instance created from that class. One class can produce many objects with different states.

4. Why do we create objects?

Answer: We create objects to represent individual entities and maintain their own state while reusing the behavior defined by a class.

5. Can we create multiple objects from a single class?

Answer: Yes. A single class can create any number of objects, for example `c1 = Car()` and `c2 = Car()`. Each object normally has its own instance attributes.

6. Where are objects stored?

Answer: Objects are stored in the memory managed by the Python runtime. In CPython, Python objects generally reside on the managed heap; variables hold references to those objects.

7. What is the purpose of the self keyword?

Answer: self is the conventional name for the reference to the current instance. It lets instance methods access that object's attributes and call its other methods.

8. Is self a keyword in Python?

Answer: No. self is not a reserved Python keyword. It is a strong convention used for the first parameter of instance methods.

9. Why is self passed as the first parameter to instance methods?

Answer: When an instance method is called as `obj.method()`, Python supplies the object as the first argument. The parameter conventionally named self receives that instance, allowing the method to work with its state.

10. Can we rename self? If yes, should we?

Answer: Yes, it can technically be renamed, such as `def show(current):`. However, self should be used because it is the universal Python convention and improves readability.

11. What are the four pillars of Object-Oriented Programming (OOP)?

Answer: The four commonly taught pillars are encapsulation, abstraction, inheritance, and polymorphism.

12. What is polymorphism? Explain with an example.

Answer: Polymorphism means one interface or operation can work with objects of different types. Example: `len()` works with strings, lists, and tuples; each type provides its appropriate behavior.

13. What is Duck Typing in Python?

Answer: Duck typing is Python's approach of focusing on what an object can do rather than its declared type. If an object provides the required method, it can often be used. The idea is summarized as: if it behaves like the required type, it can be used as such.

14. What is Dynamic Binding?

Answer: Dynamic binding means the method or behavior to execute can be determined at runtime. With overridden methods, Python looks at the actual object involved when resolving the attribute/method.

15. What is Operator Overloading? Give an example.

Answer: Operator overloading lets classes define how operators behave for their objects using special methods.

Example: defining `__add__` allows `a + b` to perform custom addition for instances.

16. What is a constructor?

Answer: A constructor is commonly described as the method involved in initializing a newly created object. In Python, `__new__` creates the instance and `__init__` initializes it; in everyday Python terminology, `__init__` is usually called the constructor.

17. What are the types of constructors in Python?

Answer: Python does not have C++-style constructor overloading. The main initialization methods are `__new__` (object creation) and `__init__` (initialization). A class may also rely on the inherited object `__init__`.

18. Can a class have multiple constructors?

Answer: Not in the traditional overloaded-constructor sense. Defining `__init__` more than once simply replaces the earlier definition. Different construction behaviors can be implemented using default arguments, `*args/**kwargs`, class methods, or other factory methods.

19. What happens if a constructor is not defined?

Answer: If `__init__` is not defined, the class inherits an initializer from its parent, ultimately object `__init__`, so an object can still be created if other requirements are satisfied.

20. What is the difference between a constructor and a normal method?

Answer: `__init__` is automatically called during normal instance construction to initialize state and must return `None`. A normal method is explicitly called and can return any appropriate value.

21. Can a constructor return a value?

Answer: No. `__init__` must return `None`. Returning another value from `__init__` raises `TypeError`. `__new__`, not `__init__`, is responsible for returning the new instance.

22. Can we call a constructor explicitly?

Answer: Yes, but it is usually unnecessary to call `__init__` directly. Normally use `ClassName(...)` so Python handles `__new__` and `__init__` correctly.

23. Can a constructor call another method?

Answer: Yes. `__init__` can call other instance methods. This is useful for organizing initialization logic, but those methods should be safe to call before all attributes are initialized.

24. What is constructor chaining?

Answer: Constructor chaining means ensuring initialization logic from parent and child classes is executed appropriately. In Python, this is commonly done with `super().__init__()` so cooperative inheritance works correctly.

25. What happens if the parent class has a parameterized constructor?

Answer: If the parent defines a required parameterized `__init__`, the child must arrange to satisfy that initialization requirement, commonly by defining its own `__init__` and calling `super().__init__(required_value)`.

26. What is encapsulation?

Answer: Encapsulation is the practice of bundling data and behavior together and controlling how internal state is accessed or modified.

27. Why do we need encapsulation?

Answer: Encapsulation helps protect invariants, reduce accidental changes, hide implementation details, and provide controlled interfaces such as methods or properties.

28. Explain public, protected, and private members.

Answer: Public members are ordinary names intended for general use. A single leading underscore indicates a protected-by-convention/internal name. A double leading underscore triggers name mangling for class attributes. Python does not enforce C++/Java-style access modifiers.

29. Does Python truly support private variables?

Answer: Python does not provide truly private instance variables in the strict access-control sense. Double underscores provide name mangling, which discourages accidental access and handles name collisions, but determined code can still access the mangled name.

30. What is Name Mangling?

Answer: Name mangling changes a name such as `__balance` inside class `Account` into a form like `__Account__balance`. It helps prevent accidental name clashes in subclasses.

31. How can we access private variables?

Answer: Technically, a double-underscore attribute can be accessed using its mangled name, e.g. `obj.__Account__balance`. However, direct access bypasses the intended abstraction; a public method or property is usually better.

32. What is the difference between encapsulation and abstraction?

Answer: Encapsulation focuses on bundling/protecting implementation state and controlling access. Abstraction focuses on exposing essential behavior while hiding unnecessary implementation details.

33. What are the advantages of encapsulation?

Answer: Advantages include better maintainability, controlled state changes, reduced coupling, clearer interfaces, easier debugging, and improved reuse.

34. What is inheritance?

Answer: Inheritance is a mechanism where a class (child/subclass) derives attributes and behavior from another class (parent/base class).

35. Why do we use inheritance?

Answer: We use inheritance to reuse behavior, model genuine 'is-a' relationships, specialize existing classes, and support polymorphism.

36. What are the different types of inheritance?

Answer: Common forms are single, multiple, multilevel, hierarchical, and hybrid inheritance. These describe how classes are related, not separate Python keywords.

37. Does Python support multiple inheritance?

Answer: Yes. Python supports multiple inheritance, where a class inherits from more than one base class.

38. What is multilevel inheritance?

Answer: Multilevel inheritance forms a chain, such as $A \rightarrow B \rightarrow C$, where `C` inherits from `B` and indirectly from `A`.

39. What is hierarchical inheritance?

Answer: Hierarchical inheritance occurs when multiple child classes inherit from the same parent class.

40. What is hybrid inheritance?

Answer: Hybrid inheritance combines multiple inheritance patterns, such as hierarchical and multiple inheritance.

41. What is the difference between inheritance and composition?

Answer: Inheritance models an 'is-a' relationship; composition models a 'has-a' relationship by placing objects inside other objects. Composition is often more flexible and reduces tight coupling.

42. What is a real-life example of inheritance?

Answer: A common example is Vehicle as a base class and Car and Bike as subclasses. A Car is a Vehicle, so shared vehicle behavior can be inherited.

43. When should inheritance NOT be used?

Answer: Avoid inheritance when there is no genuine subtype relationship, when composition is simpler, or when inheritance would create fragile coupling and complicated hierarchies.

44. Why can multiple inheritance become complicated?

Answer: Multiple inheritance can create ambiguity about which implementation should be used, especially when different parents define methods with the same name. Python resolves this through MRO, but complex hierarchies can still be difficult to understand.

45. What is Method Resolution Order (MRO)?

Answer: MRO (Method Resolution Order) is the order Python follows when searching classes for attributes and methods. It can be viewed with `ClassName.mro()` or `ClassName.__mro__`.

46. What is the Diamond Problem?

Answer: The Diamond Problem occurs when a class inherits through two paths from the same ancestor, forming a diamond. A method inherited from the common ancestor could otherwise be selected ambiguously.

47. How does Python solve the Diamond Problem?

Answer: Python solves the Diamond Problem using a consistent Method Resolution Order based on C3 linearization. cooperative `super()` calls can then traverse the hierarchy predictably.

48. Explain C3 Linearization.

Answer: C3 Linearization is the algorithm Python uses to construct MROs for multiple inheritance. It preserves local precedence order, maintains monotonicity, and produces a consistent method search order.

49. What is method overriding?

Answer: Method overriding occurs when a subclass defines a method with the same name as a method in its parent, providing specialized behavior.

50. Why is method overriding used?

Answer: Overriding allows subclasses to customize inherited behavior and is a key mechanism behind runtime polymorphism.

51. What is the difference between method overriding and method overloading?

Answer: Overriding replaces/inherits behavior in a subclass relationship. Overloading traditionally means multiple methods with the same name but different parameter signatures. Python does not support traditional compile-time method overloading by signature.

52. Does Python support method overloading?

Answer: Python does not support traditional method overloading by defining several methods with the same name. The last definition replaces earlier ones. Similar behavior can be achieved using default parameters, `*args/**kwargs`, or `singledispatch` in suitable cases.

53. What happens if a child class doesn't override a method?

Answer: If a child does not override a method, it inherits the parent's implementation according to the MRO.

54. Can constructors be overridden?

Answer: Yes. A child class can define its own `__init__`, thereby overriding the parent's initialization method. It can call the parent's initializer using `super().__init__()`.

55. Can private methods be overridden?

Answer: Python's double-underscore methods are name-mangled, so a child method with the same spelling is usually a different mangled name rather than a conventional override. Single-underscore methods can be overridden normally.

56. What is runtime polymorphism?

Answer: Runtime polymorphism occurs when the implementation selected for a method call depends on the object's actual runtime type. Method overriding and duck typing are common examples.

57. What is super()?

Answer: `super()` returns a proxy that delegates attribute/method lookup to the next class in the MRO after the current class. It is commonly used for cooperative inheritance.

58. Why do we use super()?

Answer: `super()` avoids hard-coding a particular parent class, supports cooperative multiple inheritance, and follows the class's MRO.

59. Can super() be used to access parent variables?

Answer: Yes. If a parent attribute is accessible, `super()` can access a parent implementation's attributes/methods. For class attributes, `super().attribute` may retrieve an inherited attribute according to the MRO; instance attributes normally live on `self`.

60. Can we use the parent class name instead of super()?

Answer: Yes, you can explicitly call `Parent.method(self, ...)`, but this hard-codes the parent and can interfere with cooperative multiple inheritance. `super()` is usually preferred.

61. How does super() work in multiple inheritance?

Answer: In multiple inheritance, `super()` follows the MRO rather than simply jumping to one named parent. Each cooperative class can call `super()`, allowing the chain to proceed through all relevant classes once.

62. What are the advantages of using super()?

Answer: Advantages include correct MRO-aware behavior, better multiple-inheritance support, reduced coupling to specific parent names, and easier maintenance.

63. What is abstraction?

Answer: Abstraction means exposing only the essential interface and hiding unnecessary implementation details.

64. What is the difference between abstraction and encapsulation?

Answer: Encapsulation is about bundling and controlling access to state/behavior; abstraction is about defining what an object exposes while hiding how it achieves it. They overlap but solve different design concerns.

65. What is an abstract class?

Answer: An abstract class is a class intended to define a common interface or partial implementation for subclasses. In Python it is commonly created using `ABC` and `abstractmethod` from the `abc` module.

66. Can we create an object of an abstract class?

Answer: Normally no. If a class has unimplemented abstract methods, Python prevents instantiation and raises `TypeError`.

67. What is an abstract method?

Answer: An abstract method is a method declared as required for concrete subclasses to implement, commonly using `@abstractmethod`.

68. Why do we use abstract classes?

Answer: Abstract classes enforce a common interface, make design contracts clearer, and allow shared concrete behavior while requiring subclasses to provide specific behavior.

69. Which module provides abstract class support?

Answer: The abc module provides Abstract Base Class support through ABC, ABCMeta, and abstractmethod.

70. What is the difference between an interface and an abstract class?

Answer: An interface primarily specifies a contract of operations without implementation. Python has no separate interface keyword like Java; abstract classes and protocols can serve interface-like roles. An abstract class may also contain state and implemented methods.

71. Can an abstract class have constructors, variables, and concrete methods?

Answer: Yes. An abstract class can define `__init__`, instance/class variables, properties, and concrete methods in addition to abstract methods.

72. When should we choose an abstract class?

Answer: Choose an abstract class when related subclasses share implementation/state and you want to enforce required methods. For structural typing without inheritance, typing.Protocol can often be a better fit.

73. What is an exception?

Answer: An exception is an event represented by an object that interrupts normal program flow when an error or exceptional condition occurs.

74. What is the difference between a syntax error and an exception?

Answer: A syntax error means Python cannot parse the code according to its grammar. An exception occurs during execution after the code has been successfully parsed/started.

75. What is the purpose of the try block?

Answer: The try block contains code that may raise an exception and needs exception handling.

76. What is the purpose of the except block?

Answer: The except block handles matching exceptions raised in the associated try block.

77. What is the purpose of the else block?

Answer: The else block runs when the try block completes without raising an exception.

78. What is the purpose of the finally block?

Answer: The finally block contains cleanup code that normally executes whether or not an exception occurs.

79. When does the finally block execute?

Answer: finally normally executes when leaving the try statement, including after an exception and even when return, break, or continue is used. It may not complete in cases such as abrupt process termination or certain fatal situations.

80. What is the difference between raise and assert?

Answer: raise explicitly raises an exception. assert checks a condition and raises AssertionError when the condition is false; assertions can be disabled with optimized execution, so they should not be used for essential input validation.

81. Can we write multiple except blocks?

Answer: Yes. Multiple except blocks can handle different exception types. Python checks them in order, so specific exceptions should generally come before broader ones.

82. What is the difference between Exception and BaseException?

Answer: BaseException is the root of Python's built-in exception hierarchy. Exception is its common subclass for ordinary application errors. SystemExit, KeyboardInterrupt, and GeneratorExit inherit directly from BaseException and are generally not caught by except Exception.

83. What is the difference between an Error and an Exception?

Answer: Python does not have a strict universal distinction between 'Error' and 'Exception' like some languages. Many built-in failures are exception classes, including classes whose names end in Error, such as ValueError and TypeError.

84. What is a custom exception?

Answer: A custom exception is a user-defined exception class, usually inheriting from Exception.

85. Why do we create custom exceptions?

Answer: Custom exceptions communicate domain-specific failure conditions clearly and let callers catch meaningful categories of errors.

86. What is the difference between ValueError and TypeError?

Answer: ValueError means the type is generally appropriate but the value is invalid, such as int('abc'). TypeError means an operation or function received an inappropriate type, such as '2' + 3.

87. Why should we catch specific exceptions instead of using a generic except?

Answer: Specific exceptions make handling safer and clearer. A generic except can accidentally hide programming bugs and make debugging much harder.

88. Why do we use files?

Answer: Files provide persistent storage for data so information can survive after a program ends and can be shared with other programs.

89. What are the different file opening modes?

Answer: Common modes include r (read), w (write/truncate), a (append), x (exclusive creation), plus + for updating, and b for binary mode. t denotes text mode and is the default.

90. What is the difference between r and r+?

Answer: r opens an existing file for reading only. r+ opens it for reading and writing without truncating it; the file must already exist.

91. What is the difference between w and a?

Answer: w opens for writing and truncates an existing file or creates it. a opens for writing at the end and creates the file if needed, preserving existing content.

92. What is the difference between text mode and binary mode?

Answer: Text mode reads/writes str and performs text encoding/decoding and newline handling. Binary mode reads/writes bytes and is used for images, audio, PDFs, and other raw binary data.

93. What does the seek() method do?

Answer: seek() changes the file's current position, allowing reading/writing from a specified byte/character position depending on mode and object.

94. What does the tell() method do?

Answer: tell() returns the current file position.

95. What is the difference between read(), readline(), and readlines()?

Answer: read() reads a specified amount or the remaining content; readline() reads one line; readlines() reads all lines into a list.

96. Why should we use the with statement?

Answer: The with statement automatically closes the file when the block exits, including when an exception occurs. It uses the file object's context-manager protocol.

97. What happens if a file is not closed?

Answer: If a file is not closed, buffered data may not be flushed, resources may remain occupied, and the program can eventually run into resource limits. Use with whenever possible.

98. What is buffering in file handling?

Answer: Buffering temporarily stores data in memory before reading/writing to reduce expensive system calls and improve I/O performance.

99. What is a text file?

Answer: A text file stores data interpreted as characters using an encoding such as UTF-8. Examples include .txt, .py, and many configuration files.

100. What is a binary file?

Answer: A binary file stores raw bytes without text decoding. Examples include images, executables, audio, and many document formats.

101. What is the difference between a CSV file and a TXT file?

Answer: CSV is a structured tabular text format using delimiters such as commas to represent rows and fields. TXT is a general-purpose text format without a fixed tabular structure.

102. What are the advantages of file handling?

Answer: File handling enables persistent storage, data exchange, logging, configuration storage, backups, and interaction with external data.

103. What is serialization?

Answer: Serialization converts an in-memory object or data structure into a storable/transmittable representation.

104. What is deserialization?

Answer: Deserialization reconstructs data from a serialized representation into an in-memory object or data structure.

105. Why is serialization required?

Answer: Serialization is needed when data must be saved, transmitted between processes/services, cached, or exchanged between programs.

106. What are the advantages of serialization?

Answer: Advantages include persistence, network transfer, interoperability depending on the format, caching, and convenient reconstruction of structured data.

107. What is the difference between Pickle and JSON?

Answer: Pickle is Python-specific and can serialize many Python objects, while JSON is a text-based, language-independent data format supporting a smaller set of basic data types.

108. Why is JSON preferred over Pickle in many applications?

Answer: JSON is widely preferred for APIs and interoperability because it is human-readable, supported by many languages, and easier to validate and exchange across systems.

109. What are the limitations of Pickle?

Answer: Pickle is Python-specific, can have compatibility/version considerations, and is unsafe to load from untrusted sources because unpickling can execute arbitrary code.

110. Can Pickle serialize every Python object?

Answer: No. Some objects cannot be pickled directly, especially certain runtime resources such as open file handles, sockets, and many local/closure-related objects.

111. Where is serialization commonly used?

Answer: Serialization is common in APIs, caching, databases, configuration, message queues, distributed systems, session storage, and machine-learning workflows.

112. Explain a real-world use case of serialization.

Answer: For example, an e-commerce service can serialize an order into JSON before sending it through an API. The receiving service deserializes the JSON into usable data structures. For Python-only caching, pickle may sometimes be used when its security and compatibility requirements are satisfied.

113. What is the difference between `isinstance()` and `type()`?

Answer: `isinstance(obj, Class)` checks whether an object is an instance of a class or its subclasses. `type(obj)` returns the exact object's type; `type(obj) is Class` checks exact type identity.

114. When should you use `isinstance()` instead of `type()`?

Answer: Use `isinstance()` when you want polymorphism and want subclasses to count as valid instances. Use `type(x) is T` when you specifically require the exact type.

115. What is the purpose of `issubclass()`?

Answer: `issubclass(C, B)` checks whether class C is a subclass of B, including when C is B itself. It is useful when working with class hierarchies.

116. What is the difference between `dir()` and `help()`?

Answer: `dir()` lists names/attributes available on an object (subject to Python's attribute lookup behavior). `help()` provides documentation/help information about an object, class, module, or topic.

117. How do you inspect an object's attributes and methods?

Answer: Use `dir(obj)` to discover attribute and method names, `type(obj)` to identify its type, `help(obj)` for documentation, and `obj.__dict__` when available to inspect instance/class dictionaries. `inspect` from the standard library provides more advanced introspection.